Static website generator

Calepin can build a Typst document directory as a static website. This is the same workflow used to render this documentation site: source `.typ` files live in `docs/`, and the generated `.html`, `.pdf`, and `sitemap.xml` files are written back into `docs/` for GitHub Pages.

Create a website

Use `calepin new website` to scaffold a minimal site:

```
calepin new website
```

This creates:

- `website.toml`
- `docs/index.typ`
- `docs/404.typ`

Build pages

Compile a website directory with the same `compile` command used for a single Typst document:

```
calepin compile docs public --config website.toml
```

When the input path is a directory, `--config website.toml` is required. The first positional argument is the website source directory. The optional second positional argument is the website output directory.

For GitHub Pages publishing from `docs/`, build in place by passing `docs` as both the input and output directory:

```
calepin compile docs docs --config website.toml
```

If the output directory is omitted, *Calepin* writes output beside the source files:

```
calepin compile docs --config website.toml
```

By default, each `.typ` page produces two outputs:

- `page.html`
- `page.pdf`

Use `--format html` to write only HTML:

```
calepin compile docs public --config website.toml --format html
```

Directory website builds do not support `--format pdf`, `--format png`, or `--format svg`; those formats are for single-document `calepin compile`.

Website builds use the same preprocess cache as single-document compilation. After a successful page build, *Calepin* writes a fingerprint next to the page's `results.json`. Later builds reuse that file when the chunk code, parameters, execution options, configured tools, and source-relative cache paths match. This means `make serve` does not need to re-execute expensive chunks in pages such as `example.typ` when nothing relevant changed.

Generated outputs are tracked in `.calepin/website-manifest.json`. Later builds use that manifest to remove stale generated files when pages are deleted or renamed. Full builds also scan the output directory for unexpected generated `.html` and `.pdf` files, while preserving static assets in directories such as `assets/`, `.calepin/`, `.git/`, `target/`, `node_modules/`, and `.venv/`.

Watch changes

During development, watch the website source directory:

```
calepin watch docs docs --config website.toml
```

As with `compile`, the first positional argument is the source directory and the optional second positional argument is the output directory. The initial watch build renders the whole site. After that, existing `.typ` page edits go through a fast xxh3 fingerprint lookup:

- If the changed page hash is new, *Calepin* rebuilds only that page.
- If the file was touched but the hash is unchanged, *Calepin* skips the rebuild.
- If navigation changed after an incremental rebuild, *Calepin* falls back to a full rebuild.

Calepin also falls back to a full rebuild for changes that can affect more than one page:

- `website.toml`
- theme files
- assets
- new `.typ` pages
- removed `.typ` pages
- renamed `.typ` pages
- unknown or non-page inputs

Watch and serve locally

To rebuild and serve in one command:

```
calepin watch docs docs --config website.toml --serve
```

The server injects a small reload script into HTML responses. Open browser pages poll the server and refresh after successful rebuilds.

The default bind address is `127.0.0.1:8000`. If that port is busy, choose another one:

```
calepin watch docs docs --config website.toml --serve --port 8001
```

You can also bind another interface:

```
calepin watch docs docs --config website.toml --serve --host 0.0.0.0 --port 8001
```

Serve built files

Serve an already-built static directory with:

```
calepin serve docs
```

The static server:

- serves files from the requested directory
- maps directory requests to `index.html`
- rejects path traversal
- supports `GET` and `HEAD`
- guesses content types from file extensions
- reports a clear error when the port is already in use

Use `--host` and `--port` to change the bind address:

```
calepin serve docs --host 127.0.0.1 --port 8001
```

Configuration

Website settings live in `website.toml`:

```
html_theme = "calepin-website"
title = "My Site"
description = "A static website built from Typst documents."
base_url = "https://example.com"
logo = "assets/logo.svg"
logo_alt = "My Site"
home = "index.html"
github_url = "https://github.com/user/repo"
pdf_theme = "docs/assets/pdf-theme.typ"
```

HTML theme

`html_theme` selects the HTML theme. It can be:

- `calepin-website` for the bundled website theme
- a path to a theme directory containing `layout.html`

If `html_theme` is omitted, website builds use `calepin-website`. `theme` and `template` are accepted as backward-compatible aliases.

```
html_theme = "calepin-website"
```

PDF theme

`pdf_theme` selects a Typst theme file for PDF and other paged output. If it is omitted, *Calepin* uses the bundled `calepin-pdf` theme, which styles ordinary fenced source blocks as boxes to match rendered chunk source and output blocks. You can also write `pdf_theme = "calepin-pdf"` explicitly. Set `pdf_theme = false` to disable this default. Relative paths resolve from the config file, so a theme stored with website assets can be referenced as `pdf_theme = "docs/assets/pdf-theme.typ"` when `website.toml` lives at the project root.

Metadata

`title`, `description`, and `base_url` are optional. The bundled website theme uses them for:

- browser page titles
- description metadata
- Open Graph metadata
- canonical URLs

When `base_url` is set, *Calepin* also writes `sitemap.xml`.

Branding

`logo` is a path or URL for the top-bar brand image. Relative paths are interpreted from the website output root and rewritten relative to each generated page, so `logo = "assets/logo.svg"` works from nested pages too.

`logo_alt` controls the image alt text. If no logo is configured, the bundled theme uses `title` as a text brand. `home` controls the brand link destination and defaults to `index.html`. `github_url` adds a GitHub link to the top bar.

Navigation

Navigation can be explicit:

```
[sidebar]

[[sidebar.section]]
title = "Guide"

  [[sidebar.section.item]]
  path = "index.typ"
  label = "Home"

  [[sidebar.section.item]]
  path = "usage.typ"
  label = "Usage"
```

Each section can contain any number of `item` entries. An item can point to one file with `path`:

```
[[sidebar.section.item]]
path = "install.typ"
label = "Install"
```

An item can also include files with a simple glob:

```
[[sidebar.section.item]]
glob = "guide/*.typ"
```

If no sidebar is configured, *Calepin* builds navigation from `.typ` files in the source directory. Hidden paths are skipped by default. Set `show_hidden = true` under `[sidebar]` to include hidden paths in manual navigation file discovery.

Special pages

If `404.typ` exists, *Calepin* renders it as `404.html` and `404.pdf`.

`404.typ` is excluded from automatic navigation and from `sitemap.xml`. This keeps the error page available to GitHub Pages without presenting it as a normal documentation page.

Sitemap

When `base_url` is configured, *Calepin* writes `sitemap.xml` in the output directory:

```
base_url = "https://example.com/project"
```

The sitemap is built from navigation entries. URLs are absolute and use `base_url` plus the page's generated `.html` path.

If `base_url` is removed from the config, a stale generated sitemap is removed on the next build.

Theme customization

HTML themes are MiniJinja templates. A theme directory contains a required `layout.html` file and optional `partials/`, `styles/`, and `scripts/` directories:

```
themes/my-theme/
  layout.html
  partials/
```

```
  styles/main.css
  scripts/main.js
```

Select a local theme with:

```
html_theme = "themes/my-theme"
```

Use `website.toml` for ordinary site branding and navigation changes. Create a local HTML theme when you need to change the HTML shell, top bar, sidebar, table of contents, page navigation, CSS, or JavaScript.

Website builds pass site data into HTML themes. The bundled `calepin-website` theme uses this data for sidebar navigation, previous and next page links, table of contents, metadata, the top-bar brand link, and the GitHub link. Local theme templates can access:

- `site.nav`
- `site.nav_sections`
- `site.toc`
- `site.title`
- `site.description`
- `site.base_url`
- `site.logo`
- `site.logo_alt`
- `site.home_url`
- `site.github_url`
- `site.current_url`
- `site.page_title`
- `snippets.css.code`
- `snippets.js.copy_code`
- `snippets.js.theme_toggle`
- `snippets.typst.code_block`

Bundled snippets are available to local HTML themes through the `snippets` object. For example, include reusable code/output styling and copy-button behavior with:

```
<style>{{ snippets.css.code }}</style>
<script>{{ snippets.js.copy_code }}</script>
```

PDF themes are Typst files. Their source is inserted after Calepin's executable-fence rules and before each page source, so they can add show rules for paged output without changing the original `.typ` files. Calepin also stages reusable Typst snippets under `/.calepin/snippets/typst/`; the bundled `calepin-pdf` theme imports `code-block.typ` from there. A minimal theme can replace the default source-block styling:

```
#import "/.calepin/snippets/typst/code-block.typ": code-block

#show raw.where(block: true): it => {
  if sys.inputs.at("calepin-target", default: "paged") == "html" {
    it
  } else {
    code-block(it)
  }
}
```

Source and PDF views

The bundled website theme includes a view switcher for rendered HTML, source, and PDF.

The source view is powered by a JSON script embedded in each generated HTML page. The PDF view expects the matching `.pdf` output generated by the default website build.

If you build with `--format html`, the PDF files are not generated. In that mode, the theme can still render HTML and source views, but the PDF view will not have a matching file.

Current limitations

The website generator is intentionally small. It does not yet implement:

- dependency-aware rebuilds for imported shared `.typ` files
- drafts
- taxonomies
- pagination
- search indexes
- feeds
- robots.txt generation
- Sass or SCSS compilation
- link checking
- automatic browser opening

These can be added without changing the current command shape: `new`, `compile`, `watch`, and `serve`.